Analytics & Big Data

Session 7: Machine learning II

Prof. Dr. Gerit Wagner

(2026-04-20)

Learning objectives

- **Compare** selected supervised machine learning algorithms with respect to their assumptions, flexibility, interpretability, and typical application scenarios.
- **Explain** the mechanics of selected algorithms, including distance-based classification (k-NN), recursive partitioning (decision trees), and layered weighted transformations (neural networks).
- **Select and justify** an appropriate method for a given predictive task, taking into account data characteristics, performance metrics, and trade-offs between interpretability and predictive power.

TODO

Navigation

4 Predictive Analytics

4.1 Subject of Predictive Analytics

4.2 The Analytics Process

4.3 Data Preparation

4.4 Methods, Algorithms and Applications

4.4.1 Classification

4.4.1.1 K-Nearest Neighbors

4.4.1.2 Evaluating the Quality of Classification

4.4.1.3 Decision Tree Approaches

4.4.1.4 Logistic Regression

4.4.1.5 Neural Networks

4.4.1.6 Resampling

4.4.1.7 Ensemble Learning

4.4.2 Regression

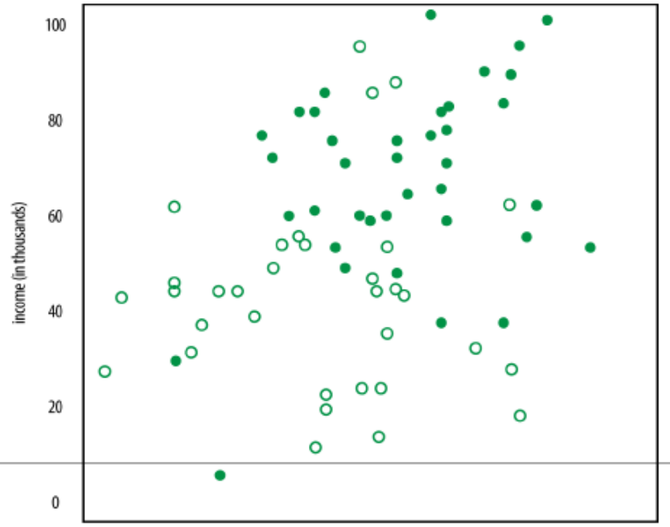
Introductory Example

Credit-Scoring is a typical example for a classification problem. A bank wants to determine the creditworthiness of a customer.

Assume you have the age, income, and a creditworthiness category of “yes” or “no” for a bunch of people and you want to use the age and income to predict the creditworthiness for a new person.

You can plot people as points on the plane and label people with an empty circle if they have low credit ratings.

What if a new guy comes in who is 49 years old and who makes 53,000 Euro? What is his likely credit rating label?



TODO

k-Nearest Neighbors

- k-Nearest Neighbors (k-NN) is an algorithm that can be used when you have a bunch of objects that have been classified or labeled in some way, and other similar objects that have not gotten classified or labeled yet, and you want a way to automatically label them.
- The intuition behind k-NN is to consider the most similar other items defined in terms of their attributes, look at their labels, and give the unassigned item the majority vote. If there's a tie, you randomly select among the labels that have tied for first.
- **Procedure of k-NN:**
  - 1. Determine parameter k (= number of nearest neighbors)
  - 2. Calculate the distances between the new object and all known labeled objects.
  - 3. Choose the k objects from all known labeled objects having the smallest distance to the new object as nearest neighbors.
  - 4. Count the frequencies of the classes of the nearest neighbors.
  - 5. Assign the new object to the most frequent class.

Measuring Similarity

The purpose of a measure of similarity is to compare two vectors of data, and compute a single number which evaluates their similarity.

Usually the similarity s(x , y) of two objects x and y is measured using the inverse of some distance measure d(x , y):

$$s(x,y) = 1 - d(x,y)$$

Euclidean distance is most often used for numerical data vectors:

$$d(x,y) = \|x - y\| = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

with n as size of the vectors.

Note: The variables must be scaled to avoid unwanted weighting resulting from different level of measurement.

Unnormalized vs. Normalized

Unnormalized:

Features / Objects	Age	Income	Rent	Years in Job
Peter	47	4000	1400	22
Thomas	33	2700	1050	15

$$d = \sqrt{(47-33)^2 + (4000-2700)^2 + (1400-1050)^2 + (22-15)^2} = 1346.38$$

Normalized:

Features / Objects	Age	Income	Rent	Years in Job
Peter	0.32	0.21	0.85	0.53
Thomas	0.26	0.13	0.54	0.34

$$d = \sqrt{(0.32-0.26)^2 + (0.21-0.13)^2 + (0.85-0.54)^2 + (0.53-0.34)^2} = 0.3771$$

Example (I)

Data:

Customer	Age	Monthly Income	Monthly Costs	Creditworthy
A	24	1800	1400	yes
B	26	1700	1300	no
C	30	2800	2200	yes
D	33	4200	3800	yes
E	35	2300	1800	no
F	39	3000	2800	no
G	41	2400	1600	yes
H	47	5000	2500	yes
I	53	4700	3200	no
J	59	2100	1700	yes

New Customer:

X	32	2900	1800	?

Example (II)

1. **k = 5 is chosen**
2. **Calculate the distances** (Normalization and then Euclidian Distance)

Customer	Age	Monthly Income	Monthly Costs	Creditworthy	Distance
A	0.0000	0.0303	0.0400	yes	0.4347
B	0.0571	0.0000	0.0000	no	0.4490
C	0.1714	0.3333	0.3600	yes	0.1726
D	0.2571	0.7576	1.0000	yes	0.8922
E	0.3143	0.1818	0.2000	no	0.2010
F	0.4286	0.3939	0.6000	no	0.4482
G	0.4857	0.2121	0.1200	yes	0.3090
H	0.6571	1.0000	0.4800	yes	0.8167
I	0.8286	0.9091	0.7600	no	0.9855
J	1.0000	0.1212	0.1600	yes	0.8096
X	0.2286	0.3636	0.2000	?	

Example (III)

3. Choose the k nearest neighbors

Customer	Age	Monthly Income	Monthly Costs	Creditworthy	Distance
A	0.0000	0.0303	0.0400	yes	0.4347
C	0.1714	0.3333	0.3600	yes	0.1726
E	0.3143	0.1818	0.2000	no	0.2010
F	0.4286	0.3939	0.6000	no	0.4482
G	0.4857	0.2121	0.1200	yes	0.3090
X	0.2286	0.3636	0.2000	?	

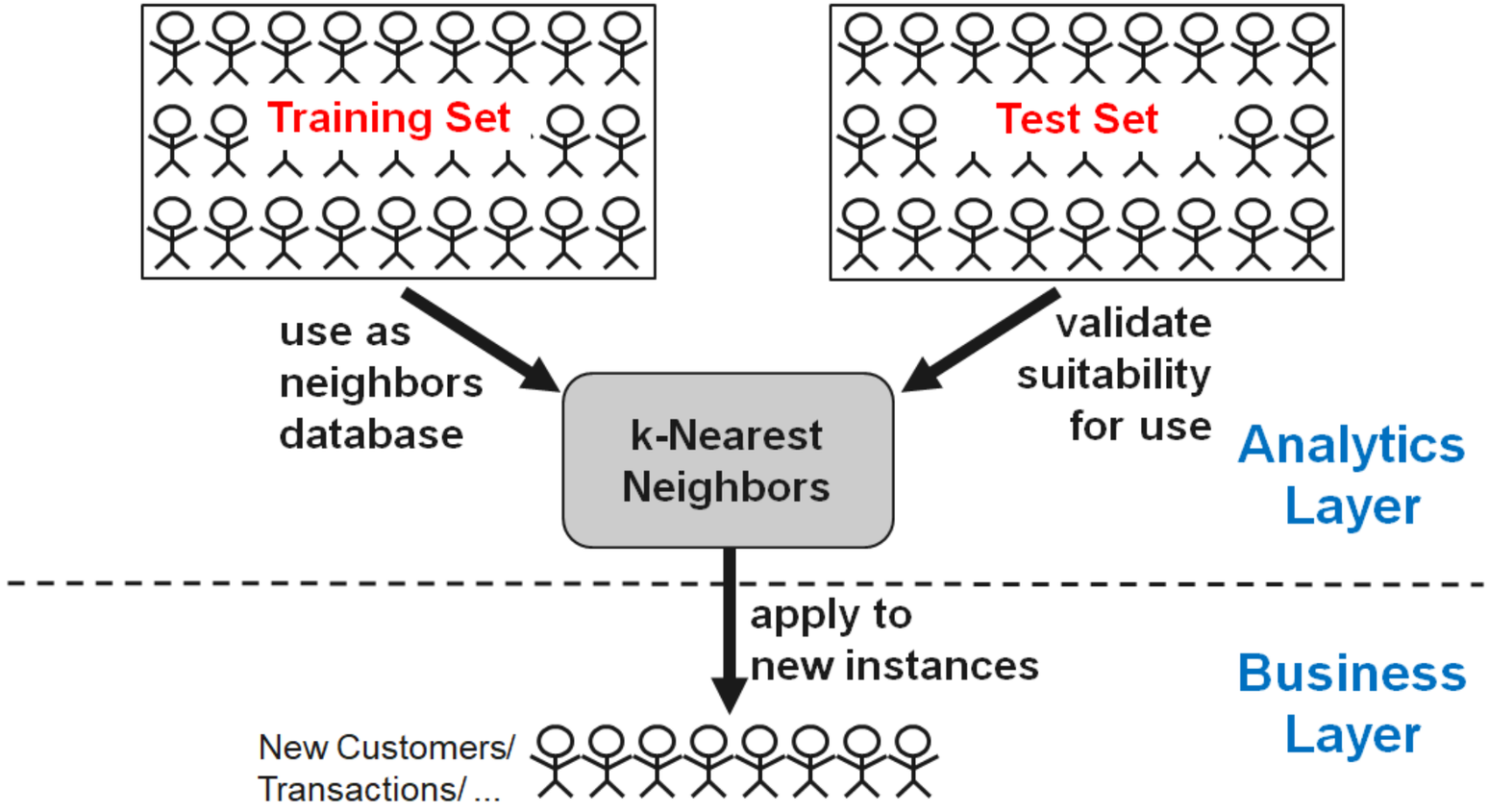
4. Count the numbers of class members

> 3 x yes; 2 x no

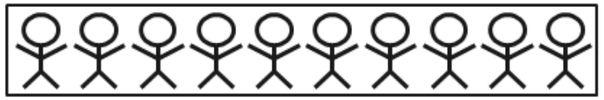
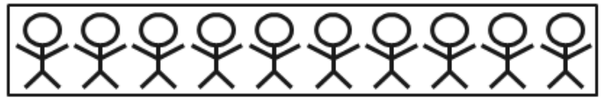
5. Assign object to most frequent class

> Customer is creditworthy!

Creation and Use of Models



Calculating Accuracies

Training Set	Test Set	
		
+ + - + - - + - + -	- + - - + + + - + -	real
+ - - + - + + - + -	- + + - + - + - + +	predicted

The training set is used to calculate the classifier model.

The accuracy of the training set indicates the goodness of fit of the model.

The test set is used to evaluate the classifier model.

The accuracy of the test set indicates the robust-ness/predictability of the model.

$$\text{Accuracy} = \frac{\#\text{Match}}{\#\text{All}}$$

Determining Parameter k

1. Split the original labeled dataset into training and test data.
2. Pick an evaluation metric. Misclassification rate or accuracy are good ones.
3. Run k-NN a few times, changing k and checking the evaluation measure.
4. Optimize k by picking the one with the best evaluation measure.

k	Accuracy
1	0.720
2	0.685
3	0.740
4	0.745
5	0.770
6	0.740
7	0.750
8	0.750
9	0.765
10	0.760

Navigation

4 Predictive Analytics

4.1 Subject of Predictive Analytics

4.2 The Analytics Process

4.3 Data Preparation

4.4 Methods, Algorithms and Applications

4.4.1 Classification

4.4.1.1 K-Nearest Neighbors

4.4.1.2 Evaluating the Quality of Classification

4.4.1.3 Decision Tree Approaches

4.4.1.4 Logistic Regression

4.4.1.5 Neural Networks

4.4.1.6 Resampling

4.4.1.7 Ensemble Learning

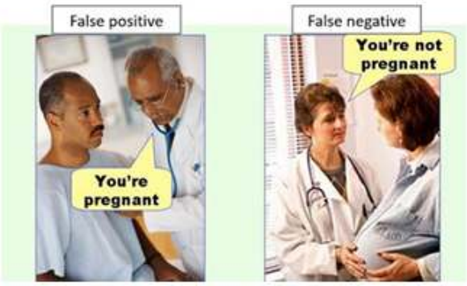
4.4.2 Regression

Evaluating the quality of Classification (I)

> True positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), are the four different possible outcomes of a single prediction for a two-class case. A false positive is when the outcome is incorrectly classified as “yes”, when it is in fact “no”. A false negative is when the outcome is incorrectly classified as negative, when it is in fact positive. True positives and true negatives are obviously correct classifications.

Confusion Matrix:

		Predicted Class	
		Yes	No
Actual Class	Yes	TP	FN
	No	FP	TN



Evaluating the quality of Classification (II)

> Test metrics are used to assess how accurately the model predicts the known values:

$$accuracy = \frac{TN + TP}{TP + FP + TN + FN}$$

$$misclassification\ rate\ = \frac{FN + FP}{TP + FP + TN + FN} = 1 - accuracy$$

> Most classification algorithms pursue to minimize the misclassification rate. They implicitly assume that all misclassification errors cost equally. In many real-world applications, this assumption is not true. Cost-sensitive learning takes costs, such as the misclassification cost, into consideration. Using costs, the error rate can be calculated via:

$$cost\ sensitive\ misclassification\ rate\ = \frac{FN \cdot C_{FN} + FP \cdot C_{FP}}{TP + FP + TN + FN}$$

Evaluating the quality of Classification (III)

Misclassification rate and accuracy can be misleading, for example in the case of imbalanced samples. Extreme case:

	yes	no
yes	1000	3
no	45	2

$$\Rightarrow \quad \frac{1000+2}{1000+45+3+2} = 0.954$$

For problems like this additional measures are required to evaluate a classifier.

Sensitivity (true positive rate, recall) measures the proportion of positives that are correctly identified as such. **Specificity** (true negative rate) measures the proportion of negatives that are correctly identified as such.

$$sensitivity = \frac{TP}{TP+FN} \qquad specificity = \frac{TN}{TN+FP}$$

Using both measures, we can compute the **Balanced Accuracy**.

TODO

Problem of Imbalancing and Accuracy

Assume the following case: A credit card company wants to create a fraud detection system to include it into their transactional systems. The outcomes should be “Accept” (Y) and “Reject” (N). Because fraud rarely occurs, the data set consists of 320 observations for Y and 139 for N. They are partitioned into training and test set. Finally, the model is trained and tested.

Because of the majority of the Y class, the training process concentrates on these cases because their correct classification promises the highest accuracy.

The results of the test of the model is consequently:

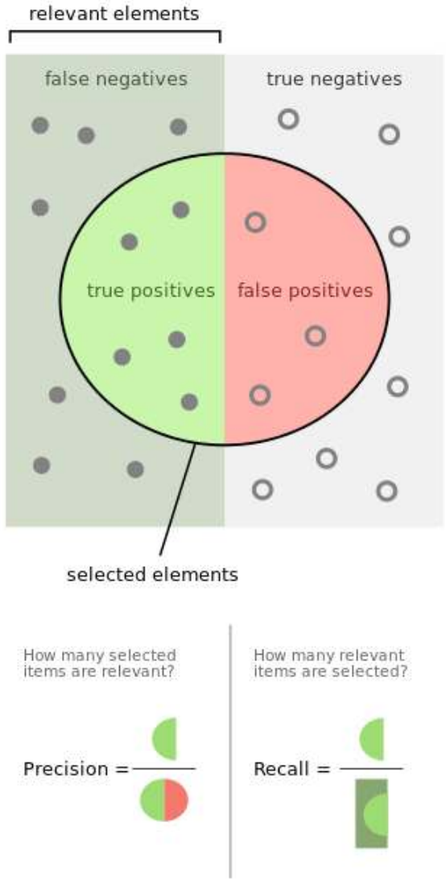
	N	Y
N	21	32
Y	6	94

Thus, the model is blind for the N cases. But these are the ones of primary interest for the company.

Evaluating the quality of Classification (IV)

Precision measures the proportion of predicted positives who are true positives. A precision of 0.5 means that whenever the model classifies a positive, there is a 50% chance of it really being a positive. The higher the precision the smaller the number of false positives.

Recall measures the percentage of positives the model is able to catch. It is defined as the number of true positives divided by the total number of positives in the dataset. A recall of 50% would mean that 50% of the positives had been predicted as such by the model while the other 50% of positives have been missed by the model.



- Source: Wikipedia

Evaluating the quality of Classification (V)

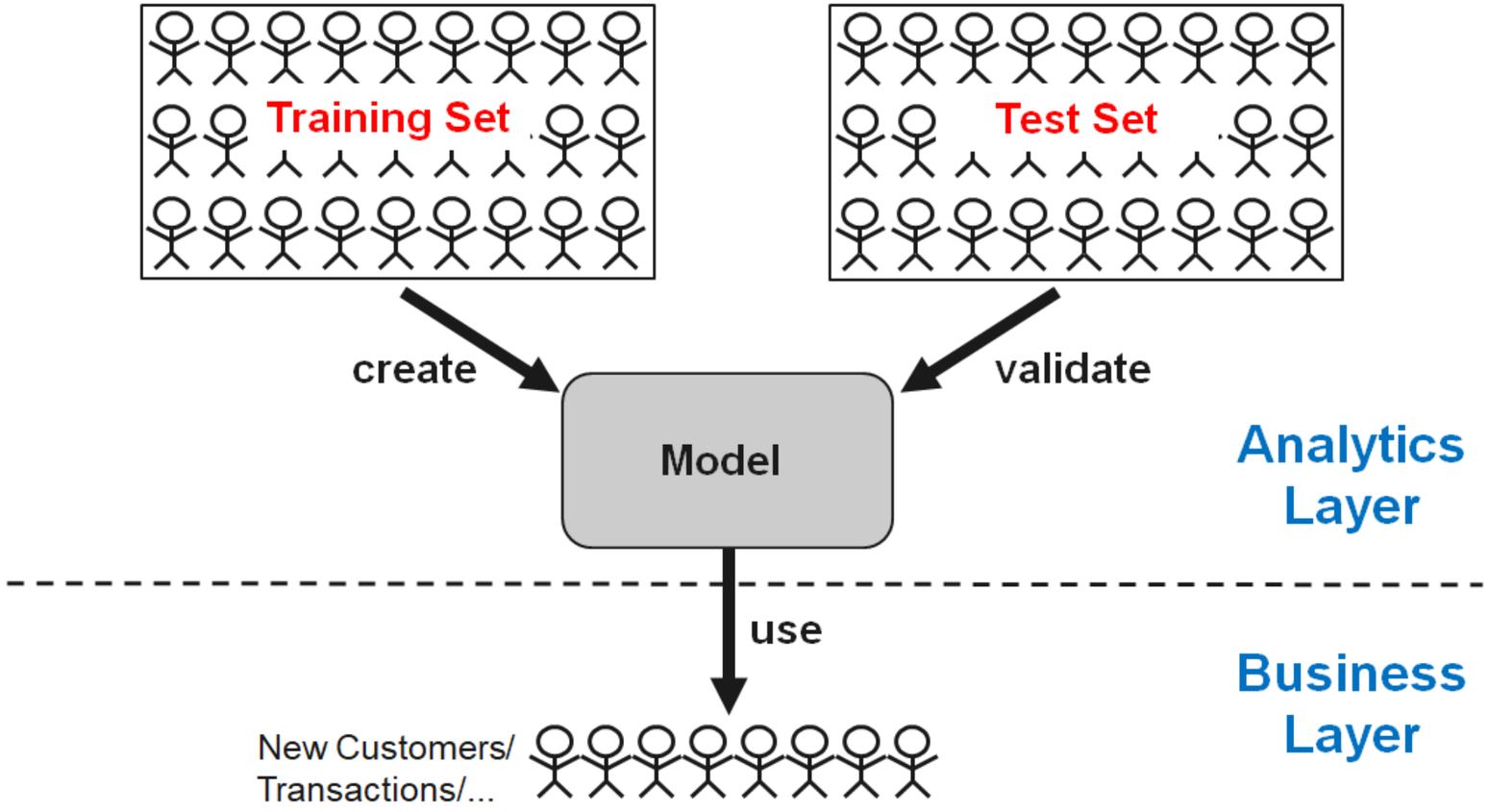
The F1 Score can be interpreted as the weighted average of both precision and recall. The main idea of the F1 Score is to strike a balance between both precision and recall and measure it in a single metric.

9 $$F1 = 2 * \frac{precision * recall}{precision + recall}$$

A F1 score reaches its best value at 1 (perfect precision and recall) and worst at 0.

It is commonly used in cases of high class imbalance.

Creation and Use of Models



Navigation

4 Predictive Analytics

4.1 Subject of Predictive Analytics

4.2 The Analytics Process

4.3 Data Preparation

4.4 Methods, Algorithms and Applications

4.4.1 Classification

4.4.1.1 K-Nearest Neighbors

4.4.1.2 Evaluating the Quality of Classification

4.4.1.3 Decision Tree Approaches

4.4.1.4 Logistic Regression

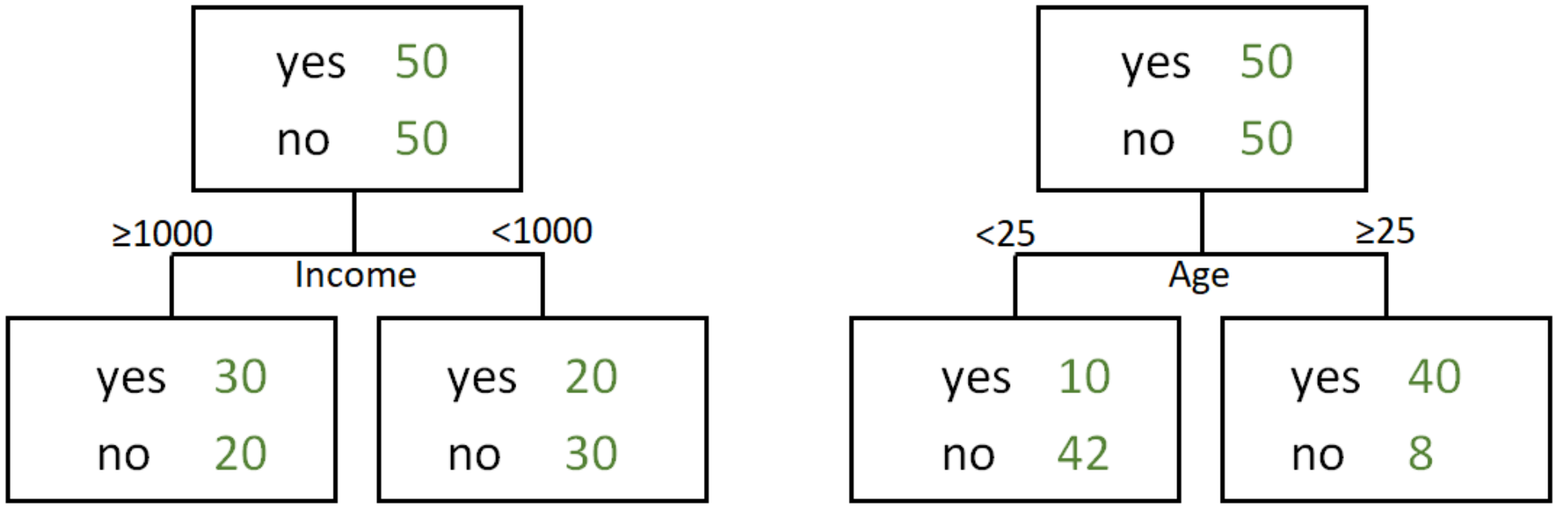
4.4.1.5 Neural Networks

4.4.1.6 Resampling

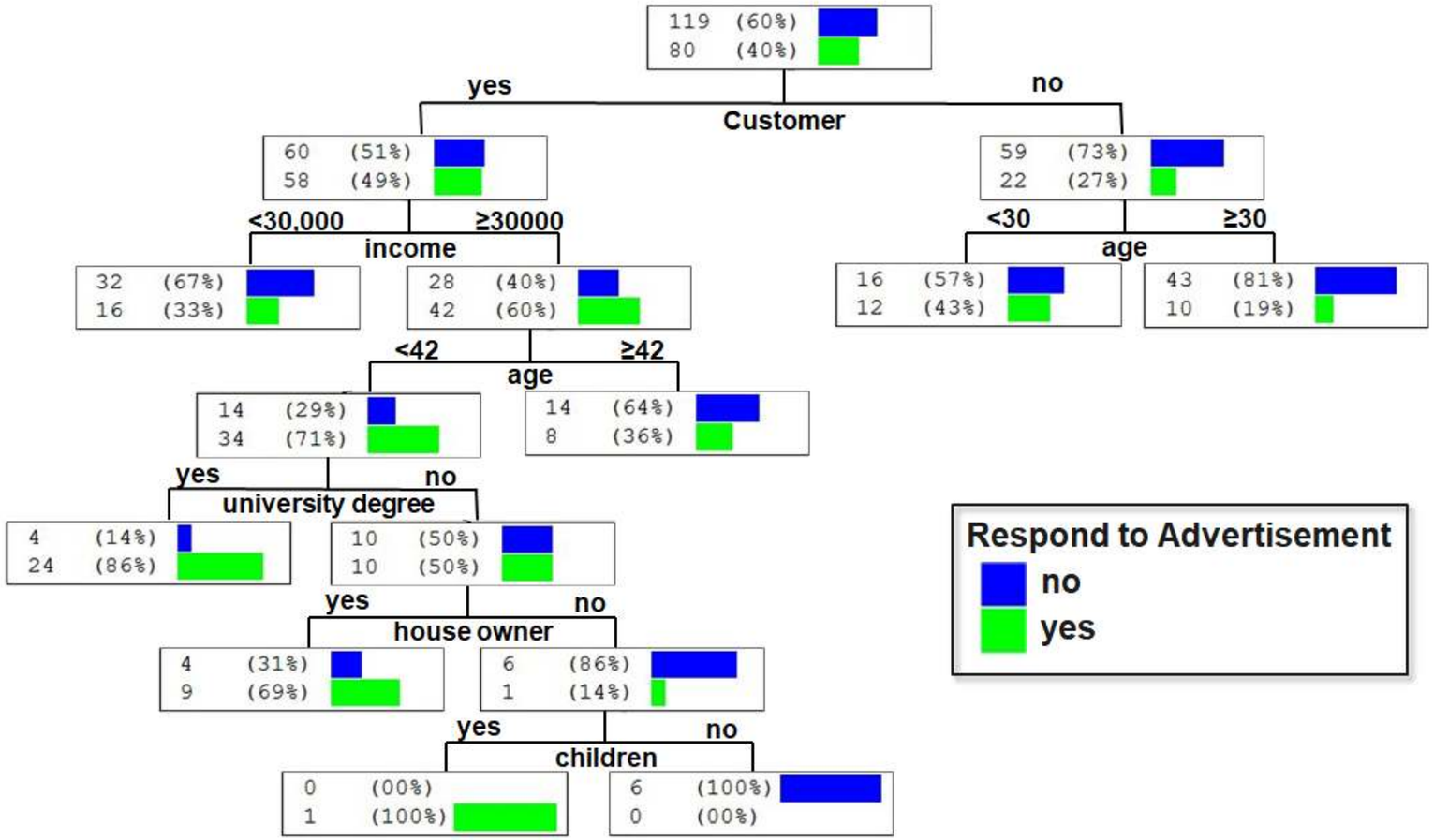
4.4.1.7 Ensemble Learning

4.4.2 Regression

Which one is better?



Introductory Example



To choose the relevant features, you can apply several methods and after this you may train a selection model using for example logistic regression. The whole process would be very time-consuming and even costly. As an alternative, you can apply a decision tree algorithm which is a stepwise or recursive classification mechanism.

Proportions in the leafs can be interpreted as probabilities.

Decision Trees (I)

Decision trees belong to the hierarchical methods of classification. They analyze step-by-step (recursive partitioning).

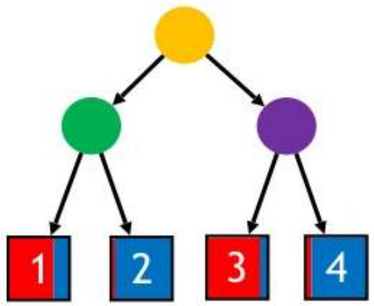
A decision tree consists of nodes and borders. The topmost node (without any parent node) is called “root”. A node without a child node is called “leaf”. Nodes that have parent and child nodes are called “interior nodes”. The interior nodes represent the splitting of the included object sets. An interior node has at least two child nodes (sons). If every interior node has exactly two child nodes, the tree is called a “binary tree”.

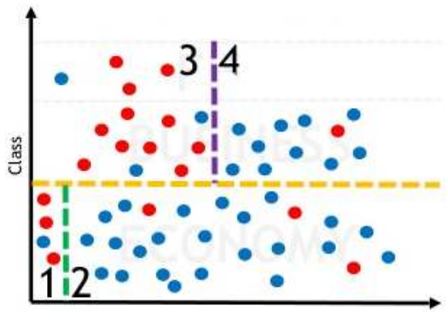
A decision tree method starts at the root, which includes all objects. The different features are compared (with an adequate measure) regarding their suitability of classification. The most appropriate feature determines the branching of the current set of objects: regarding this feature, the current set of objects is divided into disjoint subsets (partitioning). This method is now used recursively to the created child nodes (subsets).

Decision Trees (II)

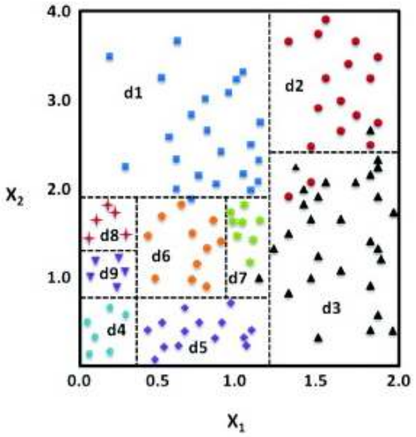
Graphically, decision tree models divide the dataspace in a large number of subspaces and search for the variables which are able to split the dataspace with the greatest homogeneity. We can think of the decision tree as a map of different path. For a distinct combination of predictor variables and their observed values, we would enter a specific path, which gives the classification in the leaf of the decision tree.

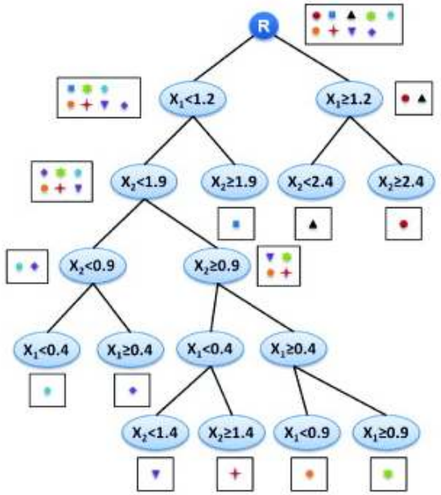
The decision tree approach does not require any assumption about the functional form of variables or distributions. Furthermore in contrast to parametric models like linear regressions, the decision tree algorithm can model multiple structures as well as complex relationships within the data, which would be difficult to replicate in a linear model.





Decision Trees (III)





— Source: http://iopscience.iop.org/article/10.1088/1749-4699/5/1/015004

Overview of important Decision Tree Methods

Name	CART	ID3	C5.0	CHAID	Random Forests
Idea	Choose the attribute with the highest information content	One of the first methods from Quinlan; uses the concept of information gain	Like ID3 based on the concept of information gain	Choose the attribute that is most dependent on the target variable	Construct many trees with different sets of features and samples (randomly). Result by voting.
Measure used	Gini-Index	Information gain (entropy)	Ratio of information gain	Chi-square split	Optional, mostly Gini-Index
Type of Splitting	Binary	Complete, pruning	Complete, pruning	Complete, pruning	Complete

Introductory Example

Day	outlook	temperature	humidity	wind	play tennis
1	sunny	hot	high	light	no
2	sunny	hot	high	strong	no
3	overcast	hot	high	light	yes
4	rainy	mild	high	light	yes
5	rainy	cold	normal	light	yes
6	rainy	cold	normal	strong	no
7	overcast	cold	normal	strong	yes
8	sunny	mild	high	light	no
9	sunny	cold	normal	light	yes
10	rainy	mild	normal	light	yes
11	sunny	mild	normal	strong	yes
12	overcast	mild	high	strong	yes
13	overcast	hot	normal	light	yes
14	rainy	mild	high	strong	no

features **class variable**

Splitting with Entropy in ID3

Day	wind	play tennis
1	light	no
2	strong	no
3	light	yes
4	light	yes
5	light	yes
6	strong	no
7	strong	yes
8	light	no
9	light	yes
10	light	yes
11	strong	yes
12	strong	yes
13	light	yes
14	strong	no

ID3 uses the entropy as a measure for the impurity of a node t:

$$\text{entropy}(t) = -\sum_{i=1}^{k} p_i \cdot \log_2 p_i$$

where p_i = relative frequency of cases in a node, that belong to a class i

Attribute Wind:

light: 6 x yes, 2 x no

$$\Rightarrow \text{entropy}(\text{light}) = -\left(\frac{6}{8} \cdot \log_2 \frac{6}{8} + \frac{2}{8} \cdot \log_2 \frac{2}{8}\right) = 0.811$$

strong: 3 x yes, 3 x no

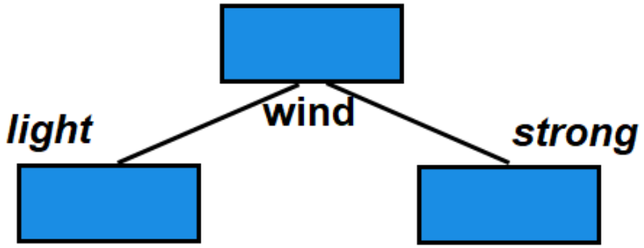
$$\Rightarrow \text{entropy}(\text{strong}) = -\left(\frac{3}{6} \cdot \log_2 \frac{3}{6} + \frac{3}{6} \cdot \log_2 \frac{3}{6}\right) = 1.0$$

Calculating the Information Gain

The information gain is a measure, that shows (by combination of the entropies) the appropriateness of an attribute for splitting:

$$\text{Information gain} = \text{entropy}(t) - \sum_{i=1}^{m} \frac{t_i}{t} \cdot \text{entropy}(t_i)$$

where m = number of values (here two: light, strong), ti = number of data sets with strong or light wind (8 resp. 6), t = total number of data sets (14) and entropy(t) = entropy before splitting.



$$\text{Information gain} = 0.94 - \left(\frac{8}{14} \cdot 0.811 + \frac{6}{14} \cdot 1.0 \right) = 0.048$$

Decision using ID3

Information gain (outlook) = 0.246

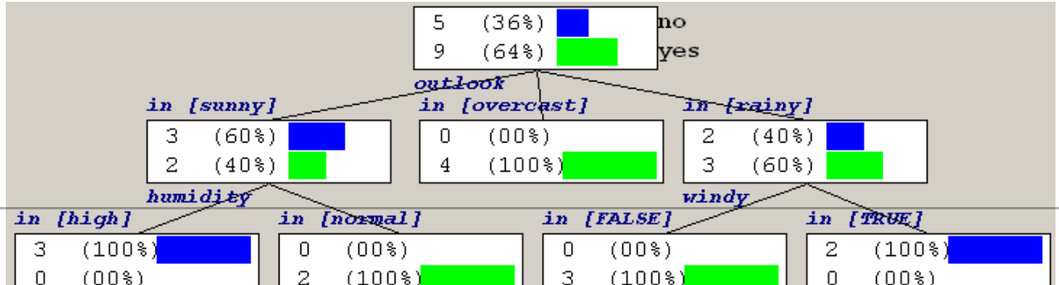
Information gain (humidity) = 0.151

Information gain (wind) = 0.048

Information gain (temperature) = 0.029

We choose the attribute with the largest information gain (here: outlook) for the first splitting.

As solution we obtain the following tree:



TODO

Decision using C5.0

ID3 tends to favor attributes that have a large number of values, resulting in larger trees. For example, if we have an attribute that has a distinct value for each record, then the entropy is 0, thus the information gain is maximal.

To compensate for this, C5.0 is a further development that uses the information gain ratio as a splitting criterion:

$$\text{Information gain ratio} = \frac{\text{entropy}(t) - \sum_{i=1}^{m} \frac{t_i}{t} \cdot \text{entropy}(t_i)}{-\sum_{i=1}^{m} \frac{t_i}{t} \cdot \log_2 \frac{t_i}{t}}$$

In the case of our example the GainRatio of Windy is

$$\text{Information gain ratio(Windy)} = \frac{0.048}{-\frac{6}{14} \cdot log_2 \frac{6}{14} - \frac{8}{14} \cdot log_2 \frac{8}{14}} = 0.049$$

and the GainRatio of Outlook is

$$\text{Information gain ratio(Windy)} = \frac{0.048}{}$$

TODO

Handling Numerical Attributes

Numerical attributes are usually splitted binary. In contrast to categorical attributes many possible splitting points exist.

The splitting point with the highest information gain is looked for. For this, the potential attribute is sorted according to its values first and then all possible splitting point and the corresponding information gains are calculated. In extreme cases there exists n-1 possibilities.

day	temperature	play
5	64	yes
6	65	no
7	68	yes
9	69	yes
11	70	yes
8	71	no
4	72	yes
14	72	no
10	75	yes
12	75	yes
2	80	no
3	81	yes
13	83	yes
1	85	no

Test the splitting point temperature = 71.5:

temperature < 71.5: 4 x yes, 2 x no

$$\Rightarrow \text{Entropy}(< 71.5) = -\left(\frac{4}{6}\cdot\log_2\frac{4}{6} + \frac{2}{6}\cdot\log_2\frac{2}{6}\right) = 0.918$$

temperature > 71.5 : 5 x yes, 3 x no

$$\Rightarrow \text{Entropy}(> 71.5) = -\left(\frac{5}{8}\cdot\log_2\frac{5}{8} + \frac{3}{8}\cdot\log_2\frac{3}{8}\right) = 0.954$$

We calculate:

$$\text{Information gain} = 0.94 - \left(\frac{6}{14}\cdot 0.918 + \frac{8}{14}\cdot 0.954\right) = 0.00143$$

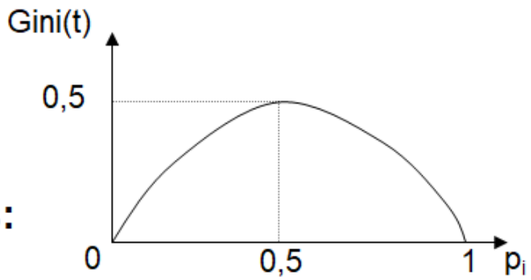
The CART Algorithm

The CART algorithm (Classification And Regression Trees) constructs trees that have only binary splits. Like C5.0, it is able to handle categorical and numerical attributes.

As a measure for the impurity of a node t, CART uses the Gini Index. In the case of two classes the Gini Index is defined as:

$$\text{Gini}(t) = 2 \cdot p_1 \cdot p_2 = 2 \cdot p_1 \cdot (1 - p_1)$$

where p_i = relative frequency of cases in a node, that belong to a class i , I = 1,2.



A node t is splitted by the attribute that minimizes:

$$\frac{t_1}{t} \cdot \text{Gini}(t_1) + \frac{t_2}{t} \cdot \text{Gini}(t_2) \rightarrow \text{Min}$$

This is equivalent to:

$$\text{Information gain} = \text{Gini}(t) - \sum_{i=1}^{2} \frac{t_i}{t} \cdot \text{Gini}(t_i) \rightarrow \text{Max}$$

Splitting in CART

Day	wind	play tennis
1	light	no
2	strong	no
3	light	yes
4	light	yes
5	light	yes
6	strong	no
7	strong	yes
8	light	no
9	light	yes
10	light	yes
11	strong	yes
12	strong	yes
13	light	yes
14	strong	no

Attribute Wind:

light: 6 x yes, 2 x no

$$\Rightarrow \text{Gini(light)} = 2 \cdot \frac{6}{8} \cdot \left(1 - \frac{6}{8}\right) = 0.375$$

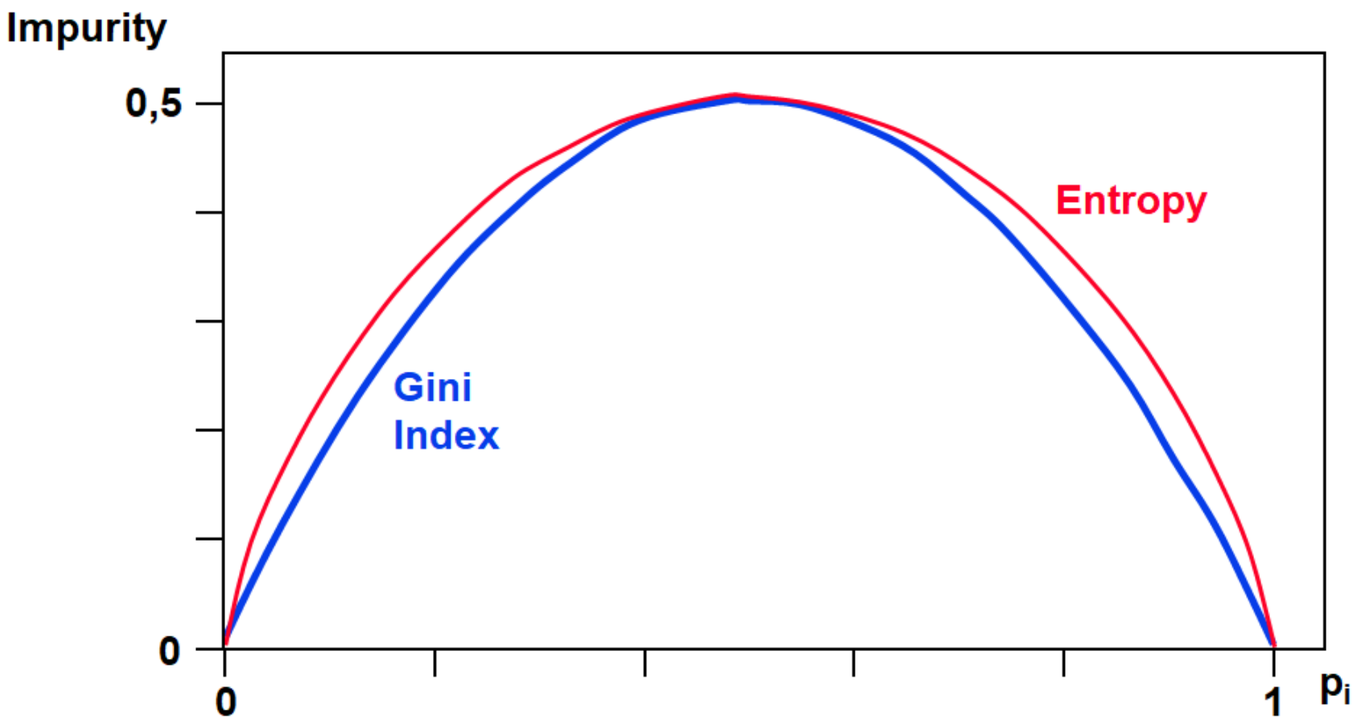
strong: 3 x yes, 3 x no

$$\Rightarrow \text{Gini(strong)} = 2 \cdot \frac{3}{6} \cdot \left(1 - \frac{3}{6}\right) = 0.5$$

resulting in:

$$\begin{aligned}\text{Information gain} &= 0.46 - \left(\frac{8}{14} \cdot 0.375 + \frac{6}{14} \cdot 0.5\right) \\ &= 0.0306\end{aligned}$$

Coherence between Entropy and Gini Index

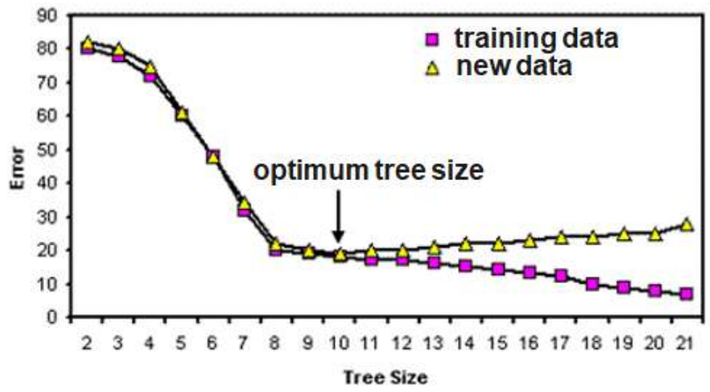


> **Remark:** Entropy has been scaled from (0, 1) to (0, 0.5)!

Overfitting (I)

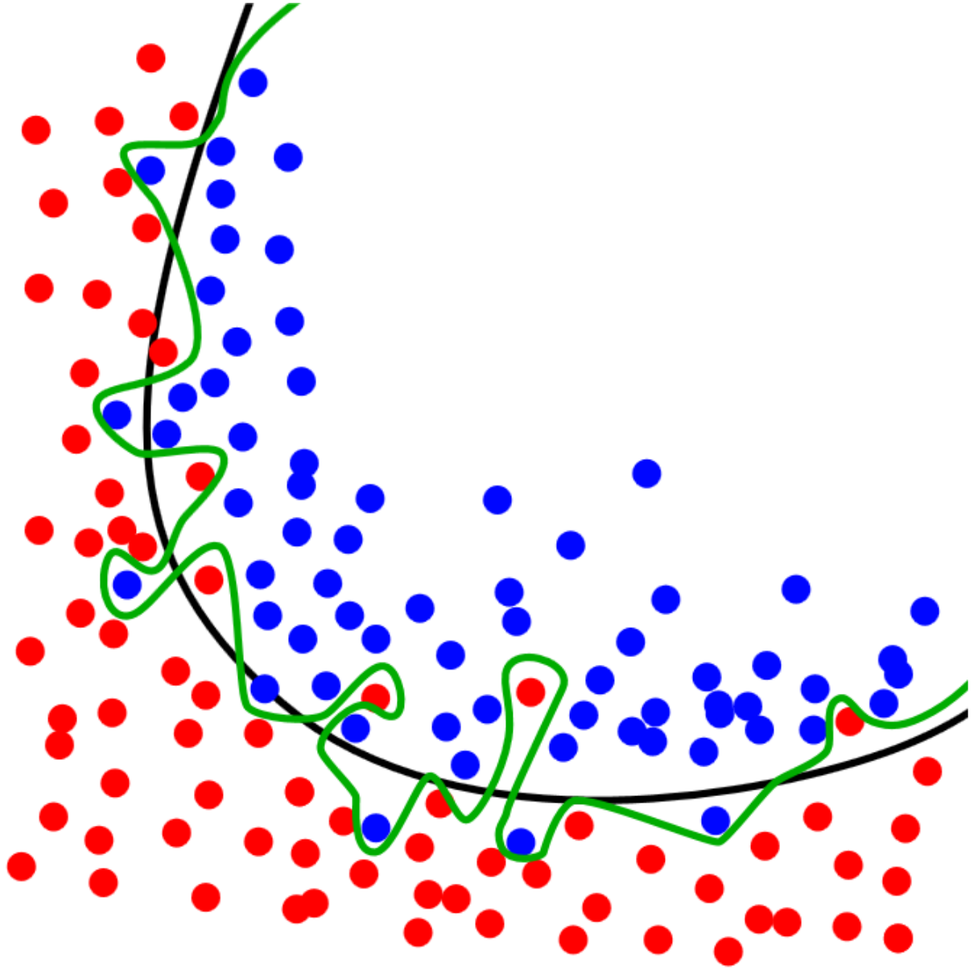
Most decision tree algorithms partition training data until every node contains objects of a single class, or until further partitioning is impossible because two objects have the same value for each attribute but belong to different classes. If there are no such conflicting objects, the decision tree will correctly classify all training objects.

If tree performance is measured from the number of correctly classified cases it is common to find that the training data gives an over-optimistic guide to future performance, i.e. with new data. A tree should exhibit generalization, i.e. work well with data other than those used to generate it. When the tree grows during training it often shows a decrease in generalization. This is because the deeper nodes are fitting noise in the training data not representative over the entire universe from which the training set was sampled. This is called ‘overfitting’.



Overfitting (II)

> The learner overfits to correctly classify the noisy data objects.

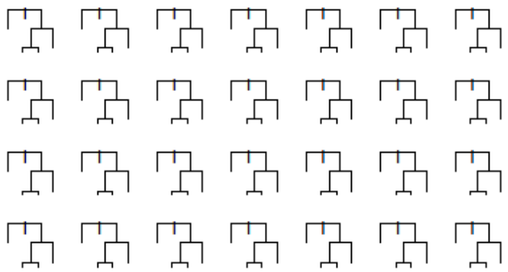


Random Forest (I)

> Random forest is an ensemble classifier that consists of many decision trees.

> For every tree a subset of the data objects and a subset of features is randomly chosen. Then the tree is constructed usually using the Gini Index.

> In the end, a simple majority vote is taken for prediction.



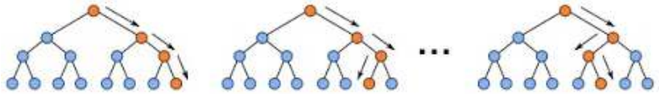
> **Algorithm:**

1. Create n samples from the original data. Frequent sample size is 2/3.
2. For each of the samples, grow a tree, with the following modification: at each node, rather than choosing the best split among all predictors, randomly sample m* of the m predictors and choose the best split from among those variables.

Random Forest (II)

> Voting-Principle of Random Forest:

> To avoid overfitting effects, the size and the depth of the trees can be restricted.



Predictive Analytics

4.1 Subject of Predictive Analytics

4.2 The Analytics Process

4.3 Data Preparation

4.4 Methods, Algorithms and Applications

4.4.1 Classification

4.4.2 Regression

Introductory Example

	A	B	C	D	E	F	G
1	URL1	URL2	URL3	URL4	URL5	URL6	URL7
2	1	0	0	0	0	0	0
3	0	0	0	0	0	0	0
4	1	0	0	0	0	0	0
5	0	1	0	1	1	0	0
6	1	0	0	0	0	0	0
7	0	0	0	0	0	0	0
8	1	0	0	0	0	0	0
9	0	0	0	0	0	1	0
10	0	0	0	0	0	0	0
11	1	0	0	0	0	0	0
12	0	0	0	0	1	0	1
13	1	0	1	0	0	0	0
14	0	0	0	0	0	0	0
15	0	0	0	0	0	1	0
16	0	0	0	0	0	0	0
17	1	0	0	0	0	0	0
18	0	0	0	0	0	0	0
19	1	0	0	0	0	0	0
20	1	0	0	0	0	0	0

BGT	BGU	BGV
URL1554	URL1555	click
0	0	0
0	0	0
0	0	1
0	1	1
0	0	0
1	0	0
0	0	1
0	0	0
0	0	1
0	0	1
0	0	0
0	0	0
0	0	1
1	0	1
0	1	1
0	0	0
0	0	0

— Source: Giant sparse matrix! A sparse matrix is a matrix in which most of the elements are zero. One matrix for every ad! Can be solved by Naïve Bayes! Looking for an alternative

TODO

Why not classical linear regression?

It is possible to implement a linear regression on such a dataset where Y={0,1}.

Problems:

- The predicted values of the linear model can be greater than 1 or less than 0.
- e is not normally distributed because Y takes on only two values.
- The error terms are heteroscedastic (the error variance is not constant for all values of X).

— Source: Bichler (2015): Course Business Analytics, TU München

Predictive Analytics

4.1 Subject of Predictive Analytics

4.2 The Analytics Process

4.3 Data Preparation

4.4 Methods, Algorithms and Applications

4.4.1 Classification

4.4.1.1 K-Nearest Neighbors

4.4.1.2 Evaluating the Quality of Classification

4.4.1.3 Decision Tree Approaches

4.4.1.4 Logistic Regression

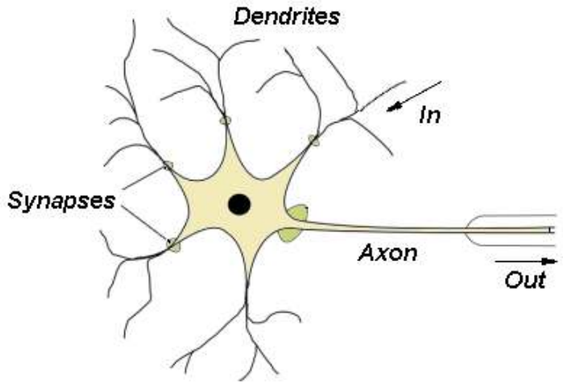
4.4.1.5 Neural Networks

4.4.1.6 Resampling

4.4.1.7 Ensemble Learning

4.4.2 Regression

Functionality of Human Neurons



An Easy Example (I)

> **f(x) = Activation function**

> e.g.

> where t = Stimulus threshold

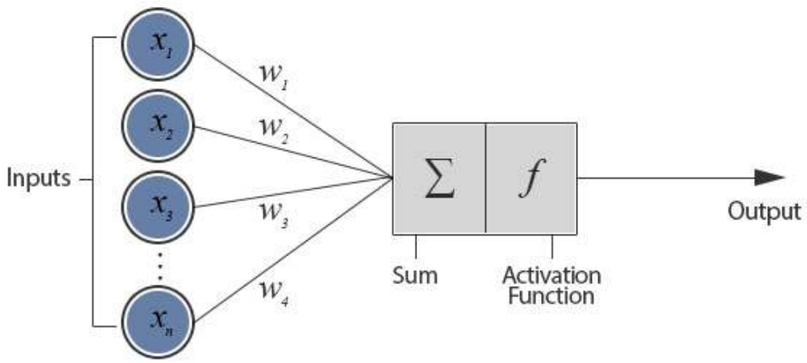
An Easy Example (II)

> **f(x) = Activation function**

> e.g.

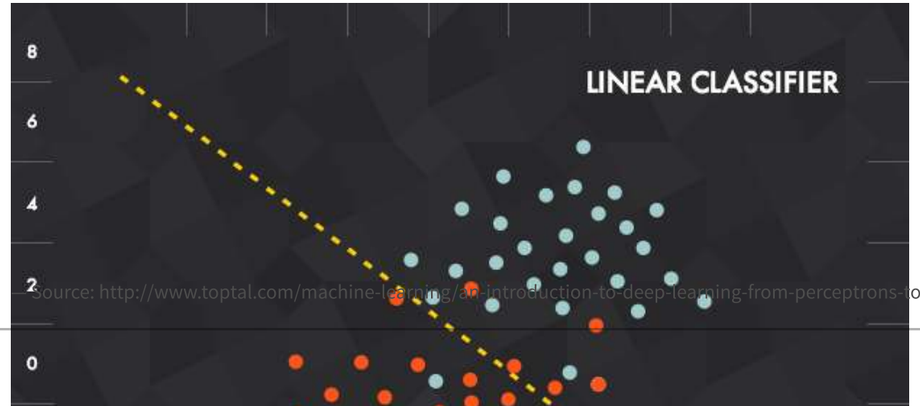
> where t = Stimulus threshold

Functionality of a Neuron



> For the case of n inputs, we can rewrite the neuron's function to

> with b = -t. b is known as the perceptron's bias. The result of this function would then be fed into an activation function to produce a labeling.



Source: http://www.toptal.com/machine-learning/an-introduction-to-deep-learning-from-perceptrons-to-deep-networks

TODO

The Multilayer Perceptron

The single perceptron approach has a major drawback: it can only learn linear functions. To address this problem, we'll need to use a multilayer perceptron, also known as feedforward neural network. Here, we add layers between the input and the output layer, so-called hidden layers. The hidden layer is where the network stores its internal abstract representation of the training data.

Input Neurons: receive signals from the outer world.

Hidden Neurons: have an internal representation of the outer world.

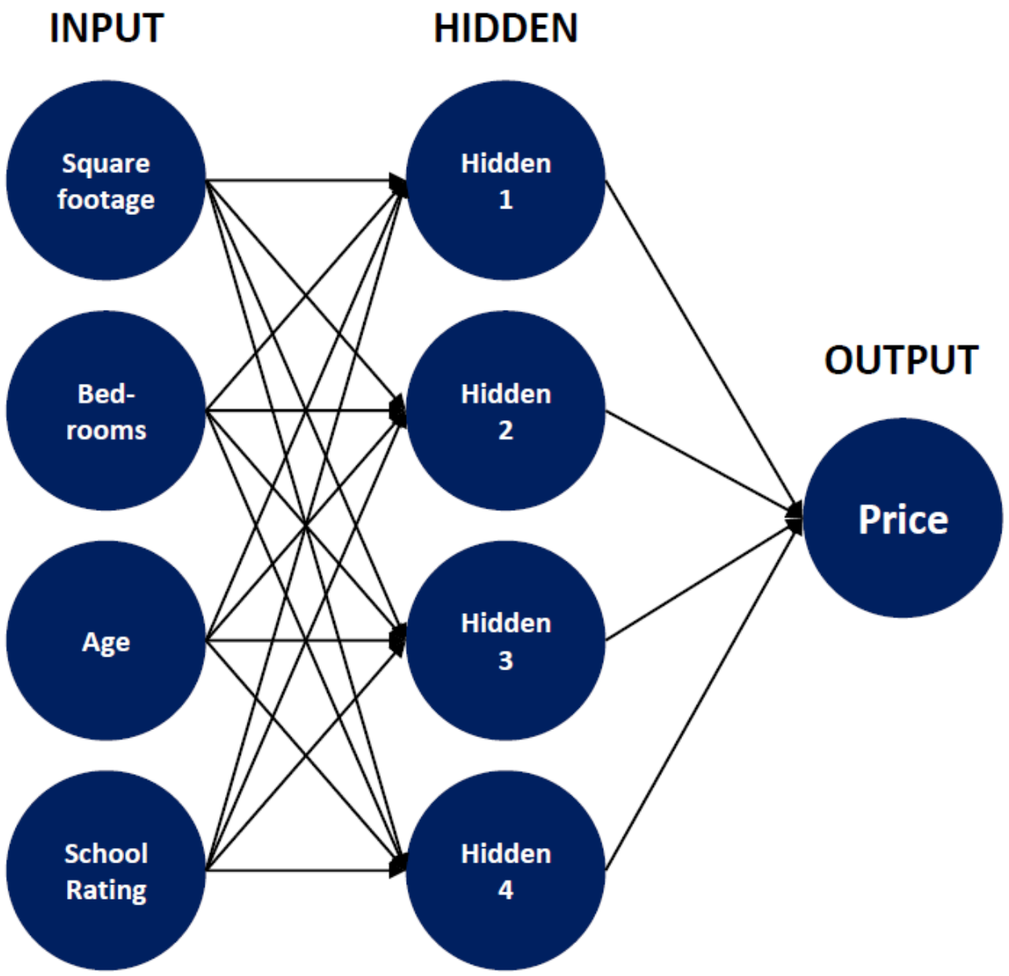
Output Neurons: pass signals to the outer world.

Types of Activation Functions

> A linear composition of linear functions is still just a linear function, so most neural networks use non-linear activation functions:

- tangens
- hyperbolicus
- logistic function (sigmoid)

Design of a Multilayer Perceptron



— Source: http://www.sclgsummit.org/uploads/presentation/8934b2d0be055a2261f5d0320f5b59bb.pdf

INPUT

— Source: http://www.sclgsummit.org/uploads/presentation/8934b2d0be055a2261f5d0320f5b59bb.pdf

TODO

Predictive Analytics

4.1 Subject of Predictive Analytics

4.2 The Analytics Process

4.3 Data Preparation

4.4 Methods, Algorithms and Applications

4.4.1 Classification

4.4.2 Regression

4.4.2.1 OLS

4.4.2.2 Ridge Regression

4.4.2.3 Support Vector Regression

4.4.2.4 Neural Networks

4.4.2.5 Decision Trees

4.4.2.6 K-Nearest Neighbors

Introductory Example

> Decision Tree for Predicting Fuel Consumption of Cars (in Miles-per-Gallon)

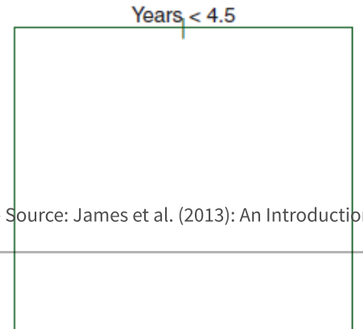
Regression Trees

Some of the tree approaches can be used for regression too. They can be used for nonlinear multiple regression. The output must be numerical.

The figure shows a regression tree for predicting the salary of a baseball player, based on the number of years that he has played in the major leagues and the number of hits that he made in the previous year.

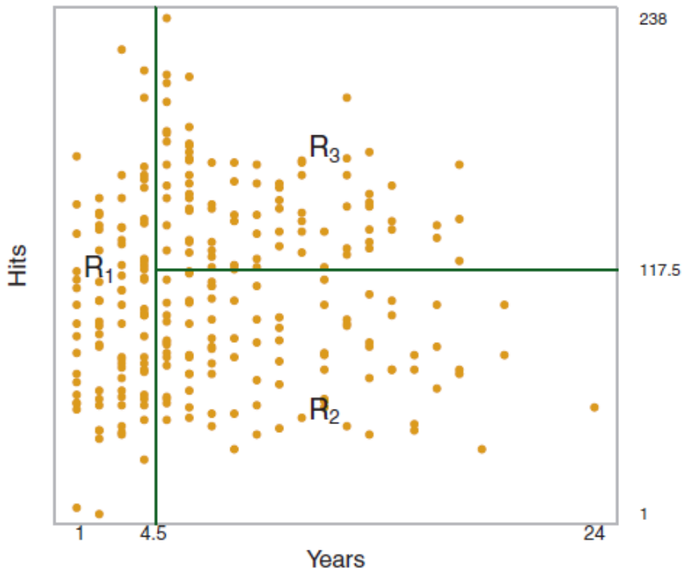
The predicted salary is given by the mean value of the salaries in the corresponding leaf, e.g. for the players in the data set with Years<4.5, the mean (log-scaled) salary is 5.11, and so we make a prediction of e^5.11 thousands of dollars, i.e. $165,670, for these players.

Players with Years>=4.5 are assigned to the right branch, and then that group is further subdivided by Hits. The predicted salaries for the resulting two groups are 1,000*e^6.00 =$403,428 and 1,000*e^6.74 =$845,346.



— Source: James et al. (2013): An Introduction to Statistical Learning with R Applications, p. 304f.

Constructing a Regression Tree (I)



— Source: James et al. (2013): An Introduction to Statistical Learning with R Applications, p. 305f.

Constructing a Regression Tree (II)

— Source: James et al. (2013): An Introduction to Statistical Learning with R Applications, p. 305f.

Random Forests for Regression

Due to the usage of means as predictors a regression tree usually simplifies the true relationship between the inputs and the output. The advantage over traditional statistical methods is, that it can give valuable insights about which variables are important and where. But the prediction ability is poor compared to other regression approaches.

A much better prediction quality can be achieved with the creation of an ensemble of trees, use them for prediction and averaging their results. This is done, when applying the Random Forests approach to a regression task.

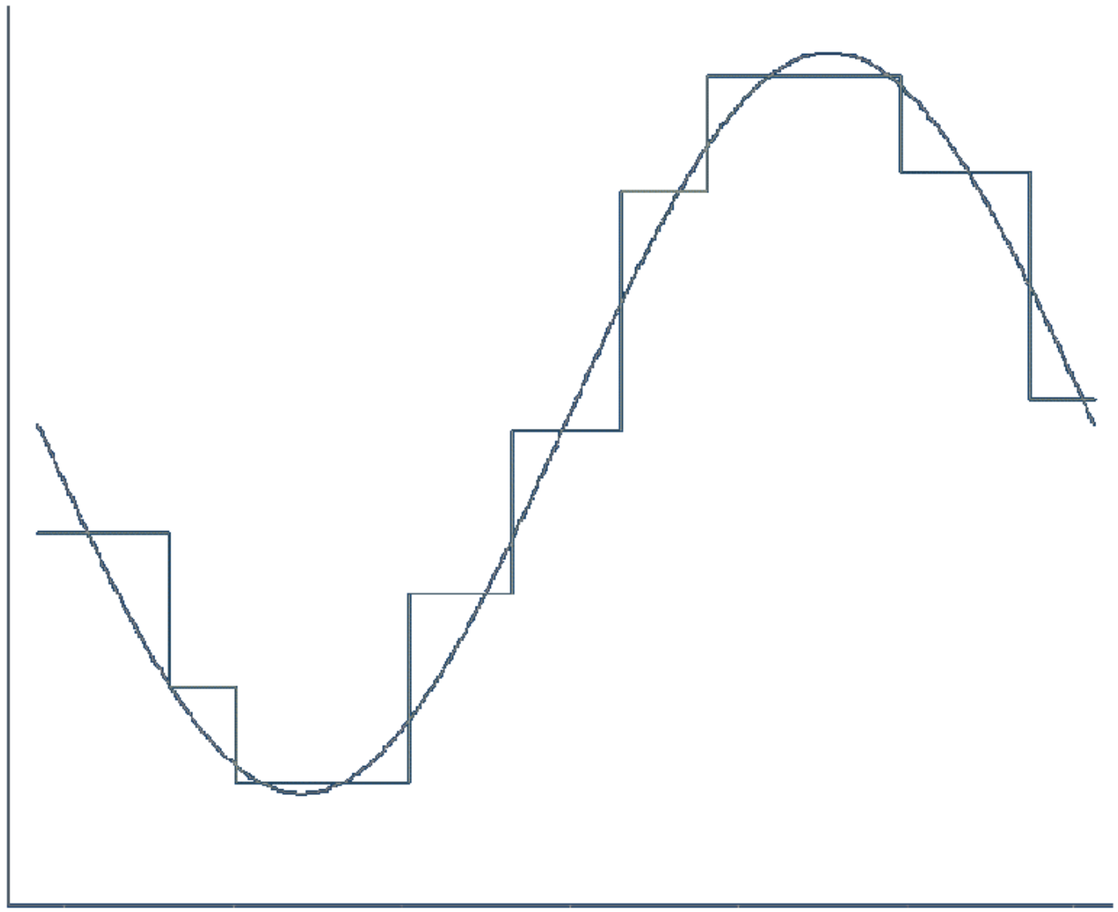
Regression Forests are an ensemble of different regression trees and are used for nonlinear multiple regression. The principle is the same as in classification, except that the output is not the result of a voting but instead of an averaging process.

The disadvantage of Random Forests is that the analysis, which aggregates over the results of many bootstrap trees, does not produce a single, easily interpretable tree diagram.

Comparing the Fitting Ability of one vs. many Regression Trees

Single Regression Tree

Average of 100 Regression Trees



Limitations of Tree Methods in Regression

When applied to regression problems, tree methods have the limitation that they cannot exceed the range of values of the target variable used in training. The reason for this lies in their design principle, how the leaves of the trees are created.

Thus, Random Forests may perform poorly when the target data is out of the range of the original training data, e.g. in the case of data with persistent trends. A solution may be a frequent re-training in this case.

An important strength of Random Forests is that they are able to perform still well in the case of missing data. According to their construction principle, not every tree is using the same features.

If there is any missing value for a feature during the application there usually are enough trees remaining that do not use this feature to produce accurate predictions.

Predictive Analytics

4.1 Subject of Predictive Analytics

4.2 The Analytics Process

4.3 Data Preparation

4.4 Methods, Algorithms and Applications

4.4.1 Classification

4.4.2 Regression

4.4.2.1 OLS

4.4.2.2 Ridge Regression

4.4.2.3 Support Vector Regression

4.4.2.4 Neural Networks

4.4.2.5 Decision Trees

4.4.2.6 K-Nearest Neighbors

k-Nearest Neighbors for Regression

> k-Nearest Neighbors cannot only be used for classification but also for regression. The only difference in regression is that the prediction is not the result of a majority vote but of an averaging process.

> A simple implementation of KNN regression is to calculate the average of the numerical target of the k-nearest neighbors. Another approach uses an inverse distance weighted average of the K-nearest neighbors. KNN regression uses the same distance functions as KNN classification.

TODO/TBD: close with ensemble classifiers?